

Cahier des Charges - Application de Suivi des Marchés des Crypto Monnaies

1. Contexte et Objectifs

Le projet consiste à développer une application pour collecter, analyser, et visualiser les données des marchés des crypto-monnaies. Le but est de fournir aux utilisateurs une plateforme qui leur permet de :

- Suivre les prix, volumes d'échange, et tendances des crypto-monnaies.
- Configurer des alertes personnalisées.
- Bénéficier de prévisions basées sur des algorithmes simples.

La finalité est de mettre en œuvre une solution robuste, en respectant les bonnes pratiques de développement logiciel, tout en utilisant des outils modernes pour garantir la qualité, la maintenabilité, et l'évolutivité.

2. Contraintes et Enjeux

Contraintes Techniques

- Collecter des données via une API publique sur [CoinCap](#).
- Utilisation de **Spring Boot** pour le backend.
- Base de données pour stocker les données (MySQL).
- Application web en **Thymeleaf** pour le frontend.
- Dockerisation et orchestration avec **Kubernetes**.
- Notifications par email via un service comme **SMTP** (Gmail).

Contraintes Organisationnelles

- Le projet est réalisé en **solo**, ce qui implique une organisation stricte pour respecter les délais.
- Livraison finale et maintenance prévue pour le **10 janvier 2025**.
- Méthodologie **Agile (Scrum)** adoptée pour organiser le travail avec des sprints de 2 semaines.

3. Périmètre Fonctionnel

3.1 Application Console

- Collecte automatique des données toutes les X minutes via une API.
- Stockage des données en base (prix, volumes, variations).
- Logs des opérations pour assurer un suivi précis des exécutions.

3.2 Application Web

- **Visualisation des données :**
 - Graphiques interactifs : courbes de prix, des graphiques en chandeliers ou heatmaps.
 - Navigation intuitive entre crypto monnaies.
 - Filtrage par plage de temps.
- **Alertes personnalisées :**
 - Définition des seuils (prix, variations).
 - Notifications par email.
- **Prévisions :**
 - Calculs basés sur des moyennes mobiles, régressions linéaires, etc.
 - Affichage des marges d'erreur.

4. Outils et Technologies

4.1 Développement

- Backend : **Spring Boot** (avec prise en charge de HTTPS).
- Frontend : **Thymeleaf**
- Base de données : **SQLite** (choix par simplicité et légèreté).
- API de données : **CoinCap**.
- Tests unitaires : **JUnit**, **Mockito**.
- couverture de test: **SonarQube**
- Tests de performance : **k6**.
- Tests de sécurité : **OWASP ZAP**

4.2 Intégration et Déploiement

- Versionnement : **GitHub**.
- CI/CD : **GitHub Actions**
- Dockerisation : **Docker**.
- Orchestration : **Kubernetes** (Minikube pour développement local).

4.3 Monitoring

- **Prometheus** et **Grafana** pour suivre la performance et l'état de l'application.

4.4 Gestion de Projet

- Méthodologie Agile : **Scrum** avec sprints de 2 semaines.
 - Outils : **Jira** pour le backlog produit et les tâches.
-

5. Organisation et Planification

5.1 Étapes du Projet

Étape	Durée	Description
1. Étude du projet et début	18 nov. -18 nov.	Compréhension du projet, récupération d'un clé API, choix technologiques, écriture du cahier des charges, architecture du projet
2. Analyse et Conception	20 nov. -29 nov. 2024	Création des interfaces principales et définition des tests unitaires à réaliser avec des mocks. Configuration initiale de SonarQube.
3. Développement Console	30 nov. -11 déc. 2024	Implémentation de l'application console : collecte et stockage des données. Tests des comportements des interfaces avec Mockito. Mise en place du pipeline CI/CD avec GitHub actions pour chaque pull request.
4. Développement Backend	12 déc.-22 déc. 2024	Création des endpoints REST pour exposer les données. Validation des intégrations avec des tests unitaires et d'intégration. Suivi des performances avec SonarQube.
5. Développement Frontend	23 déc. -1 janv. 2025	Interface web pour la visualisation et la configuration des alertes. Ajout des tests fonctionnels pour les endpoints consommés par le frontend.
6. Test et intégration	2 janv. -6 janv. 2025	Exécution des tests de performance et de sécurité (k6, OWASP ZAP). Déploiement Docker/Kubernetes. Validation finale de la couverture des tests avec SonarQube.
7. Documentation et soutien	7 janv. - 9 janv. 2025	Finalisation de la documentation (technique et utilisateur). Rapport sur la qualité du code et la couverture des tests. Préparation de la soutien.

5.2 Livrables

- Code source versionné sur **GitHub** avec plusieurs commits.
- Documentation technique :

- Schémas de base de données, API, architecture.
 - Instructions pour lancer le projet (Docker, Kubernetes).
- Documentation utilisateur :
 - Guide pour utiliser l'application (console et web).
- Rapport de tests (unitaires, mocks, intégrations, performance, sécurité).
- Tableau de bord Agile (captures Jira).
- Pipeline CI/CD opérationnel.

6. Architecture Logicielle

L'architecture suit un modèle **3 tiers** :

1. **Frontend** :
 - Application web interactive (Thymeleaf).
2. **Backend** :
 - API REST en Spring Boot pour récupérer les données de la base et gérer les alertes.
3. **Base de données** :
 - Stockage des données collectées et des configurations des utilisateurs.

L'application sera dockerisée et déployée sur Kubernetes pour garantir l'évolutivité et la modularité.

7. Gestion des Risques

Risques Techniques -> Solutions

- API publique non fonctionnelle ou limitée → Prévoir des alternatives (API secondaires).
- Difficultés liées à l'intégration Docker/Kubernetes → Répartir la tâche dans un sprint séparé.
- Complexité des tests de sécurité/performance → Prioriser les cas critiques.

Risques Organisationnels -> Solutions

- Charge de travail élevée (seul sur le projet) → Organisation stricte et respect des sprints.
- Retards possibles → Intégrer des marges dans le planning.